

JavaScript Asynchronous Programming

Notes

1. Synchronous vs Asynchronous Functions

Synchronous function: Executes line by line, blocking further code until it completes.

```
function syncFunction() {  
  console.log("This is a synchronous function");  
}
```

Asynchronous function: Declared with the `async` keyword. It can run other code without waiting, and always returns a `Promise`.

```
async function asyncFunction() {  
  console.log("This is an asynchronous function");  
  syncFunction();  
  console.log("This is after the synchronous function");  
}  
  
asyncFunction();
```

- Even though it's called “asynchronous,” a plain `async` function with no `await` inside still runs its body synchronously, top to bottom.
- The real asynchronous behavior kicks in only when `await` is used inside it.

2. The Call Stack

The **call stack** is a LIFO (Last In, First Out) structure that JavaScript uses to keep track of function calls.

```
function one() {  
  return 1;  
}  
  
function two() {  
  return one() + one();  
}  
  
function three() {  
  let ans = two() + one();  
  console.log(ans);  
}  
  
three();
```

How it executes:

- `three()` is pushed onto the stack.
- Inside `three()`, `two()` is called → pushed onto the stack.
- Inside `two()`, `one()` is called twice → each pushed and popped after returning.

- two() returns 2 (1 + 1), then is popped off the stack.
- Back in three(), one() is called again → returns 1.
- ans = 2 + 1 = 3.
- console.log(ans) prints 3.
- three() is popped off the stack — stack is now empty.

Key idea: Functions are pushed onto the stack when called and popped off when they return. JavaScript is single-threaded, so only one thing runs on the stack at a time.

3. Promises

A **Promise** is an object representing the eventual completion (or failure) of an asynchronous operation.

Promise states:

State	Meaning
Pending	Initial state, neither fulfilled nor rejected
Fulfilled	Operation completed successfully
Rejected	Operation failed

Creating a Promise

```
function saveToDB(data) {
  return new Promise((success, failure) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    console.log(internetSpeed);
    if (internetSpeed > 5) {
      success("Success: Data saved successfully");
    } else {
      failure("Failure: weak internet connection");
    }
  });
}
```

- **success** (conventionally **resolve**) is called when the operation succeeds.
- **failure** (conventionally **reject**) is called when the operation fails.

Consuming a Promise: .then() and .catch()

- **.then()** runs when the promise is **fulfilled**.
- **.catch()** runs when the promise is **rejected**.

```
let request = saveToDB("Hello World");

request
  .then(() => {
    console.log("Data saved successfully");
    return saveToDB("A");
  })
  .then(() => {
```

```
        console.log("Data2 saved successfully");
        return saveToDB("B");
    })
    .then(() => {
        console.log("Data3 saved successfully");
    })
    .catch(() => {
        console.log("Weak internet connection");
    });
```

Key idea — Promise chaining: Returning a new promise inside a `.then()` lets the next `.then()` wait for it, creating a sequential chain. If any promise in the chain rejects, control jumps straight to `.catch()`.

4. `fetch()` and `async / await`

`fetch()` is used to make HTTP requests (e.g., to an API) and returns a Promise.

`async` / `await` is syntactic sugar over promises — it lets asynchronous code read like synchronous code.

```
async function getData() {
  try {
    let response = await fetch("https://api.github.com/users");
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.log(error);
    console.log("Weak internet connection");
  } finally {
    console.log("Finally block");
  }
}

getData();
```

Breakdown:

- `await` pauses execution of the async function until the awaited promise settles (without blocking the rest of the program).
 - `fetch(url)` returns a promise that resolves to a Response object.
 - `response.json()` parses the response body as JSON — this is also asynchronous, so it needs `await` too.
 - **try...catch...finally:**
 - `try` — code that might throw/reject.
 - `catch` — runs if an error occurs (e.g., network failure).
 - `finally` — always runs, regardless of success or failure (good for cleanup, like hiding a loading spinner).
-

5. Query Strings

A **query string** is the part of a URL that sends extra data to the server, starting with `?`.

```
search?q=apple
```

```
https://www.google.com/search?q=apple
```

- `?` marks the start of the query string.
- `q=apple` is a key-value pair (key = value).
- Multiple parameters are separated by `&`:

```
https://example.com/search?q=apple&sort=price&limit=10
```

- Common uses: search terms, filters, pagination, tracking parameters, API request parameters.
- Can be read in JavaScript using the `URLSearchParams` API:

```
const params = new URLSearchParams(window.location.search);  
console.log(params.get("q")); // "apple"
```

Quick Summary Table

Concept	Purpose
Sync function	Runs immediately, blocks until done
Async function	Returns a Promise, enables non-blocking code
Call Stack	Tracks function execution order (LIFO)
Promise	Represents a future value: pending -> fulfilled/rejected
<code>.then()</code> / <code>.catch()</code>	Handle promise success / failure
<code>async</code> / <code>await</code>	Cleaner syntax for working with promises
<code>try/catch/finally</code>	Error handling around async code
<code>fetch()</code>	Makes HTTP requests, returns a Promise
Query string	Sends key-value data via URL after <code>?</code>